SIMATS ENGINEERING

ASSESSMENT TOOL-2

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA1708

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 HOUR

QUESTIONS: 5 **Total Marks: 50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I __JASMITHA R(192524295)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

JASMITHA R

Scenario 1: AI Drone Delivery in a Flood-Affected Region

i. Behaviour of Greedy Best-First Search

Greedy Best-First Search selects the next node purely on the heuristic value h(n) – here, the straight-line distance to the destination – while ignoring the cost already incurred, g(n). In this scenario, the drone would always move toward the location that appears geometrically closest to the target.

Strengths: very fast decisions, low memory usage, and a simple computation that suits the urgency of an emergency delivery.

Weaknesses: it is neither complete nor optimal. Because it ignores actual travel cost, it can be misled by a route that looks close on a straight line but is blocked or dangerous due to weather, terrain, or a flooded road. Under uncertainty, this can lead the drone into dead ends or high-risk paths, causing delay rather than avoiding it.

ii. Behaviour of A* Search

A* uses the evaluation function f(n) = g(n) + h(n), combining the real cost accumulated so far with the estimated remaining cost. This means the drone weighs both the effort/risk already spent and the projected distance to the goal. If the heuristic is admissible (never overestimates true cost), A* is guaranteed to find the optimal route – the one with true minimum cost, not just apparent closeness. It naturally accounts for terrain and weather penalties already encountered along the path, giving a more balanced and safer decision than pure greedy search.

iii. Impact of Heuristic Accuracy

For Greedy Best-First Search, the heuristic is the only factor driving decisions, so an inaccurate heuristic directly produces poor or unsafe routes. For A*, the heuristic only influences search efficiency as long as it stays admissible – optimality is preserved even with a weaker (but admissible) heuristic, only at the cost of exploring more nodes. If the heuristic overestimates true cost (becomes inadmissible), A* can lose its optimality guarantee just like greedy search, so heuristic design matters more for A* in terms of speed, and critically for Greedy search in terms of correctness.

iv. Effect of Dynamic Changes (Blocked Paths)

When a road becomes blocked mid-flight, both algorithms must replan. Greedy search reacts quickly by simply re-aiming at the next closest-looking option, but this reactive behaviour can repeatedly walk into new obstacles since it never reconsiders accumulated cost. A* replanning is more computationally expensive because it must re-evaluate path costs, but it produces higher-quality alternate routes. In fast-changing environments, full A* replanning at every update may be too slow, which is why real-time variants such as D* Lite or Anytime A* are typically used to keep the benefits of A* while meeting strict time constraints.

v. Recommendation

A* (or a real-time adaptation such as D* Lite/Anytime A*) is the recommended choice. It offers a strong balance between optimality and safety, since it accounts for real terrain and weather cost rather than raw distance alone. Pure greedy search can be reserved only as a fast fallback when computation time is critically constrained, accepting a trade-off in route quality.

Scenario 2: Smart City Traffic Signal Optimization

Problem formulation: This is an optimization problem, not a path-finding problem. The state is a configuration of signal timings across all intersections; the objective function combines waiting time, fuel consumption, and congestion (to be minimized). Traditional path-search algorithms such as BFS or A* are inefficient here because there is no single defined goal state or path to a goal – the state space is enormous and continuous/combinatorial (each intersection has many possible timing values), and the "best" configuration is a matter of optimizing a cost surface rather than reaching a destination.

i. Hill Climbing in This Scenario

Hill Climbing would start from an initial timing configuration and repeatedly make small adjustments (e.g., changing one intersection's green-light duration) that improve the objective function, stopping when no neighbouring change helps. Its limitations are: it only looks at immediate neighbours with no backtracking, it is highly sensitive to the starting configuration, and it offers no guarantee of finding the best overall (global) timing plan, especially in a landscape as large and constantly shifting as city-wide traffic.

ii. Local Maxima, Plateaus, and Ridges

Local maxima: a signal configuration where no single small adjustment improves traffic flow, even though a very different configuration elsewhere would perform much better – e.g., a set of timings that looks good locally but ignores a better city-wide rhythm.

Plateaus: flat regions where changing a signal's timing produces almost no measurable difference in congestion, causing the search to wander without clear progress – for instance, adjusting a light by a few seconds during low-traffic hours.

Ridges: situations where improvement requires simultaneously adjusting several correlated intersections together (a coordinated "diagonal" move); a hill-climber that changes one variable at a time cannot see this joint benefit and gets stuck.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing allows the search to occasionally accept a worse configuration, with this tendency decreasing over time (as the "temperature" cools), which lets it escape local maxima and plateaus that would trap Hill Climbing. Genetic Algorithms instead maintain a population of many candidate timing plans simultaneously and use selection, crossover, and mutation to explore the search space broadly and in parallel, making them naturally resistant to getting stuck and well-suited to large, combinatorial spaces like city-wide signal timing.

iv. Recommendation

A Genetic Algorithm, or a hybrid of Genetic Algorithm with Simulated Annealing (and increasingly, reinforcement learning for continuous adaptation), is recommended. This choice suits the scale of the problem (many intersections, huge search space) and its need for adaptability, since traffic patterns change dynamically throughout the day and population-based or annealing-based methods can be re-run or continuously adapted without being trapped by local optima.

Scenario 3: Autonomous Mars Rover

i. Why an Online Search Agent Is Needed

Offline search requires complete knowledge of the environment before any action is taken, so a full plan can be computed in advance. The rover, however, does not have a complete map of the Martian terrain and cannot fully sense hazards from a distance. An online search agent interleaves computation with action: it takes a step, senses the environment, updates its knowledge, and decides the next step, which is essential when the environment is unknown and must be discovered incrementally.

ii. Challenges of Partial Observability and Dynamic Environments

Partial observability means the rover's sensors have limited range and cannot detect hazards (craters, loose rocks) hidden behind obstacles or beyond sensor reach, which risks unsafe moves based on incomplete information. Dynamic conditions – such as dust storms, shifting sand, or newly exposed obstacles – mean that even previously mapped areas may become unreliable, forcing the rover to continuously re-verify and replan rather than trusting a static internal map.

iii. Building and Updating the Internal Model

The rover incrementally builds an internal representation of the terrain (similar to an occupancy grid or belief-state map) as it moves and senses its surroundings. Each new sensor reading updates this model – marking discovered hazards, confirming safe regions, and revising previous assumptions – so the rover's plan is continuously refined using the most current knowledge rather than committed to a single fixed map.

iv. Online Search vs. Uninformed and Informed Search

Uninformed search (e.g., BFS/DFS) would require exhausting large portions of an unknown, effectively infinite terrain with no guidance, making it impractical. Classical informed search (e.g., offline A*) needs an accurate map in advance to compute heuristic-guided optimal paths, which the rover simply does not have. Online search agents, such as Learning Real-Time A* (LRTA*), combine heuristic guidance with the ability to learn and revise estimates as new information is discovered, making them the only practical fit for this exploration problem.

v. Recommendation

An online, informed search approach such as LRTA* is recommended. It provides heuristic-guided efficiency while remaining adaptive to unknown and changing terrain, allowing the rover to balance safety (avoiding unnecessary risk near unexplored hazards) with steady progress toward its sampling goals.

Scenario 4: University Exam Timetabling as a CSP

i. Variables, Domains, and Constraints

Variables: each exam/subject to be scheduled (e.g., $Exam_1$, $Exam_2$, …, $Exam_n$).

Domains: the set of possible (time slot, exam hall) combinations available for each exam, e.g., {Mon 9AM–Hall A, Mon 2PM–Hall B, Tue 9AM–Hall A, …}.

Constraints: (1) no student may have two exams scheduled at the same time – a binary constraint between any two exams sharing at least one student; (2) hall capacity/availability – an exam's assigned hall must be free and large enough at that slot; (3) precedence constraints – certain subjects must be scheduled before others; (4) faculty availability – the invigilating faculty must be free at the assigned slot; (5) special accommodations – some students/exams require specific rooms or extended time, modelled as additional unary or binary constraints on those exam variables.

ii. How Backtracking Search Works Here

Backtracking search assigns a time-slot/hall value to one exam at a time and checks it against all constraints involving previously assigned exams. If the assignment is consistent, the search moves to the next exam; if it violates a constraint (e.g., creates a student clash or exceeds hall capacity), the search backtracks to the previous exam and tries a different value. This continues, depth-first, until every exam has a valid assignment or the search proves no solution exists with the current ordering.

iii. Constraint Propagation (Forward Checking)

Forward checking looks ahead after each assignment: whenever an exam is scheduled, it removes now-invalid slot options from the domains of all exams still unassigned that share a constraint with it (e.g., removing that time slot from every exam sharing a common student). This detects failures much earlier than plain backtracking, since a variable left with an empty domain signals an immediate dead end, saving the search from exploring doomed branches. Stronger propagation such as arc-consistency (AC-3) can prune domains even further before search begins.

iv. Real-World Challenges and Best Strategy

As the number of courses and students grows, the search space expands exponentially, and plain backtracking becomes too slow for practical use. The best strategy combines backtracking with constraint propagation (forward checking or AC-3) and strong variable/value ordering heuristics: the Minimum Remaining Values (MRV) heuristic to pick the most constrained exam next, the Degree heuristic to break ties by choosing exams involved in the most constraints, and Least Constraining Value (LCV) to choose slot options that leave the most flexibility for remaining exams. For very large universities, local search techniques such as min-conflicts can be layered on top to refine an initial (possibly imperfect) timetable quickly, giving the system the scalability it needs.

Scenario 5: Strategic Game AI – Minimax and Alpha-Beta Pruning

i. How Minimax Models the Problem

Minimax represents the game as a tree of alternating layers: MAX nodes (the AI's turn, trying to maximize the outcome) and MIN nodes (the opponent's turn, assumed to play optimally to minimize the AI's outcome). Starting from the leaves (terminal or cutoff states with an evaluated utility), values are propagated upward – MAX nodes take the maximum of their children's values, MIN nodes take the minimum – until the root determines the move that guarantees the best outcome for the AI assuming a perfectly rational opponent.

ii. Computational Challenges of Large Game Trees

The number of nodes in a game tree grows as roughly b^d, where b is the branching factor (number of legal moves per state) and d is the search depth. For real-time strategy games with many possible actions

per turn, this leads to a combinatorial explosion that makes searching to a useful depth within the game's time limit computationally infeasible without optimization.

iii. How Alpha-Beta Pruning Improves Efficiency

Alpha-beta pruning tracks two bounds while traversing the tree: alpha, the best value MAX can currently guarantee, and beta, the best value MIN can currently guarantee. Whenever a branch is found where beta ≤ alpha, the remaining sibling branches cannot affect the final decision and are pruned without being evaluated, since the opponent (or the AI) would never allow that branch to be reached. With good move ordering, this can reduce the effective branching factor from b to roughly √b, changing the time complexity from O(b^d) to O(b^(d/2)) in the best case – allowing the same time budget to search roughly twice as deep.

iv. Designing an Evaluation Function

Since full-depth search is infeasible, the tree is cut off at a limited depth and an evaluation function estimates the desirability of that intermediate state. A well-designed evaluation function combines multiple weighted factors relevant to the game – for example, material/resource advantage, positional strength or map control, unit mobility, and safety/threat exposure. These factors are combined into a single weighted score, with weights typically tuned through testing, self-play, or machine learning so the function reliably approximates the true minimax value without requiring a full search.

v. Strategy for Balancing Optimality and Real-Time Performance

The recommended approach is iterative deepening combined with alpha-beta pruning: the AI searches to depth 1, then 2, and so on, always having a usable move ready and simply stopping when the time limit is reached. This is enhanced with move ordering (e.g., trying previously good moves first, such as the killer-move heuristic) to maximize pruning, and transposition tables to avoid re-evaluating repeated states. Paired with a well-tuned evaluation function at the cutoff depth, this strategy gives the AI a strong, near-optimal decision within strict real-time constraints.